

Yes. Let's make this **code-first**, because for Apex asynchronous work, understanding the syntax and being able to write the class + test class is the important part.

We'll go **one mechanism at a time**. We'll start with **Future Methods**, and I'll build from a very simple question to a realistic Salesforce question.

1. Future Method — Code From Zero

First, memorize the basic structure

A Future Method looks like this:

```
apex

public class MyFutureClass {

    @future
    public static void myFutureMethod() {

        // Code that should run asynchronously

    }

}
```

There are several keywords here, and **every one has a reason**.

2. Understand every keyword

```
public
```

```
apex
```

```
public class MyFutureClass
```

Means the class can be accessed from outside its immediate context.

For your beginner coding exercises, you'll commonly write:

```
apex
```

```
public class ...
```

class

apex

```
public class MyFutureClass
```

You're creating an Apex class.

The class is basically a container for your methods.

@future

This is the most important keyword.

apex

```
@future
```

It tells Salesforce:

"Execute this method asynchronously."

Without it:

apex

```
public static void myMethod()
```

would be a normal synchronous method.

With it:

apex

```
@future  
public static void myMethod()
```

it becomes a Future Method.

public

Again:

apex

```
public static void myFutureMethod()
```

makes the method accessible.

```
static
```

Future methods **must be static**.

So this:

apex

```
@future  
public static void myFutureMethod()
```

is correct.

Don't write:

apex

```
@future  
public void myFutureMethod()
```

for a Future Method.

```
void
```

apex

```
public static void myFutureMethod()
```

means the method doesn't return a value.

Future methods must return `void`.

So don't write:

apex

```
@future  
public static String myFutureMethod()
```

3. Your first Future Method

Let's say the question is:

Create a Future Method that prints "Hello from Future Apex" in the debug log.

The answer:

```
apex

public class MyFutureClass {

    @future
    public static void myFutureMethod() {

        System.debug('Hello from Future Apex');

    }

}
```

That's it.

4. How do I call this method?

You don't call it differently from a normal static method.

```
apex

MyFutureClass.myFutureMethod();
```

For example, Execute Anonymous:

```
apex

MyFutureClass.myFutureMethod();
```

Salesforce puts the Future Method into the asynchronous processing queue.

Conceptually:

```
Execute Anonymous
    ↓
myFutureMethod()
    ↓
Salesforce puts it in async queue
    ↓
Future executes
```

5. Testing a Future Method

This is **very important**.

For asynchronous Apex, Salesforce gives us:

apex

```
Test.startTest();  
Test.stopTest();
```

These two are extremely important.

Your test:

apex

```
@isTest  
public class MyFutureClassTest {  
  
    @isTest  
    static void testFutureMethod() {  
  
        Test.startTest();  
  
        MyFutureClass.myFutureMethod();  
  
        Test.stopTest();  
  
    }  
}
```

Let's understand **every single line**.

6. Test class line by line

Line 1

apex

```
@isTest
```

This tells Salesforce:

```
"This is a test class/method."
```

Test code isn't production code.

Salesforce uses it to verify that your Apex works correctly.

Line 2

apex

```
public class MyFutureClassTest
```

We're creating a test class.

Usually:

Original class:

```
MyFutureClass
```

Test class:

```
MyFutureClassTest
```

Line 3

apex

```
@isTest
```

This tells Salesforce the following method is a test method.

Line 4

apex

```
static void testFutureMethod()
```

This is the test method.

Again:

- `static` → test methods are static
 - `void` → doesn't return anything
 - `testFutureMethod` → method name
-

7. The most important part

apex

```
Test.startTest();
```

Think of this as:

"Start the actual test execution now."

It gives the code inside the block a fresh governor-limit context and, importantly for asynchronous Apex, marks the point where the async operation is initiated.

Then:

apex

```
MyFutureClass.myFutureMethod();
```

calls the Future Method.

But remember:

It doesn't execute like normal synchronous code.

It gets queued.

Then:

apex

```
Test.stopTest();
```

is extremely important.

It tells Salesforce:

"Finish the asynchronous work that was started inside the test."

So:

apex

```
Test.startTest();

MyFutureClass.myFutureMethod();

Test.stopTest();
```

means:

```
START TEST
  ↓
Call Future Method
  ↓
Future gets queued
  ↓
STOP TEST
  ↓
Future executes
```

★ Memorize this

Whenever you test asynchronous Apex:

apex

```
Test.startTest();

// call asynchronous code

Test.stopTest();
```

8. But this test isn't actually testing anything!

Correct.

Our first example only proves that the method can execute.

A useful test should verify a **result**.

Let's make a proper Salesforce example.

9. Example Question #1

Question

Create a Future Method that accepts an Account Id and updates the Account's Description to "Processed asynchronously" .

This is a much more realistic question.

10. Step 1 — Create the Future Class

```
apex

public class AccountFutureProcessor {

    @future
    public static void processAccount(Id accountId) {

        Account acc = [
            SELECT Id, Description
            FROM Account
            WHERE Id = :accountId
        ];

        acc.Description = 'Processed asynchronously';

        update acc;
    }
}
```

Now let's understand this **line by line**.

11. Line-by-line explanation

Line 1

```
apex

public class AccountFutureProcessor {
```

Creates our Apex class.

Class name:

```
AccountFutureProcessor
```

We're putting our Future Method inside it.

Line 2

```
apex  
  
@future
```

Makes the following method asynchronous.

Line 3

```
apex  
  
public static void processAccount(Id accountId) {
```

There are several things here.

```
public
```

Method can be accessed.

```
static
```

Required for a Future Method.

```
void
```

Future methods don't return a value.

```
processAccount
```

Our method name.

```
Id accountId
```

This is our parameter.

We're passing the Account's Id.

For example:

```
001XXXXXXXXXXXXXX
```

12. Why pass the Id instead of the Account?

This is an important Future Method concept.

Don't do:

apex

```
@future
public static void processAccount(Account acc)
```

Instead:

apex

```
@future
public static void processAccount(Id accountId)
```

Then query the Account inside the Future Method:

apex

```
Account acc = [
    SELECT Id, Description
    FROM Account
    WHERE Id = :accountId
];
```

Why?

Because the Future Method runs later.

The record could change between:

Calling Future Method

and

Future Method actually executing

So we pass the Id and retrieve the latest record when the Future executes.

13. Understand the SOQL

apex

```
Account acc = [  
    SELECT Id, Description  
    FROM Account  
    WHERE Id = :accountId  
];
```

This means:

Find the Account whose Id equals the Id we passed into the Future Method.

This:

apex

```
:accountId
```

means:

Use the Apex variable `accountId` inside the SOQL query.

14. Update the record

apex

```
acc.Description = 'Processed asynchronously';
```

We're changing the Account's Description.

Then:

apex

```
update acc;
```

performs DML.

So:

```
Query Account  
↓  
Change Description  
↓  
update acc
```

15. Now write the Test Class

Here's the complete test:

apex

```
@isTest
public class AccountFutureProcessorTest {

    @isTest
    static void testProcessAccount() {

        Account acc = new Account(
            Name = 'Test Account'
        );

        insert acc;

        Test.startTest();

        AccountFutureProcessor.processAccount(acc.Id);

        Test.stopTest();

        Account updatedAcc = [
            SELECT Id, Description
            FROM Account
            WHERE Id = :acc.Id
        ];

        System.assertEquals(
            'Processed asynchronously',
            updatedAcc.Description
        );
    }
}
```

Now let's break this down.

16. Creating Test Data

apex

```
Account acc = new Account(
    Name = 'Test Account'
);
```

We create an Account **inside the test**.

This is important.

Tests generally should create the data they need rather than relying on existing org data.

Then:

```
apex  
  
insert acc;
```

puts it into Salesforce.

Now:

```
acc.Id
```

contains the newly created Account's Id.

17. Start asynchronous testing

```
apex  
  
Test.startTest();
```

Now we're starting the important section of our test.

18. Call the Future Method

```
apex  
  
AccountFutureProcessor.processAccount(acc.Id);
```

We pass:

```
apex  
  
acc.Id
```

to:

apex

```
processAccount (Id accountId)
```

So conceptually:

```
acc.Id  
  ↓  
processAccount (acc.Id)  
  ↓  
accountId receives acc.Id
```

19. Stop Test

apex

```
Test.stopTest();
```

This causes the Future Method to finish executing within the test context.

So after:

apex

```
Test.stopTest();
```

we can query the Account and verify the result.

20. Query the updated Account

apex

```
Account updatedAcc = [  
    SELECT Id, Description  
    FROM Account  
    WHERE Id = :acc.Id  
];
```

We retrieve the Account again.

Why?

Because we want to see whether the Future Method actually changed it.

21. Assert the result

This is one of the most important concepts in test classes.

apex

```
System.assertEquals(  
    'Processed asynchronously',  
    updatedAcc.Description  
);
```

This means:

"I expect `updatedAcc.Description` to equal `'Processed asynchronously'.`"

Expected value:

Processed asynchronously

Actual value:

apex

`updatedAcc.Description`

If they're equal:

✓ Test passes.

If not:

✗ Test fails.

22. Your complete mental flow

You should be able to visualize the entire test:

```
CREATE TEST ACCOUNT  
    ↓  
INSERT ACCOUNT  
    ↓  
Test.startTest()  
    ↓  
Call Future Method  
    ↓
```

```
Future Method gets Account Id
      ↓
Future Method queries Account
      ↓
Changes Description
      ↓
Updates Account
      ↓
Test.stopTest()
      ↓
Query Account again
      ↓
assertEquals()
      ↓
PASS / FAIL
```

23. Example Question #2 — Future Method with multiple records

Now let's make it slightly more realistic.

Question:

Create a Future Method that accepts a list of Account Ids and changes the Description of every Account to "Processed" .

This teaches you something very important:

Future methods can accept collections of primitive values such as `List<Id>` .

Apex code

```
apex

public class AccountFutureProcessor {

    @future
    public static void processAccounts(List<Id> accountIds) {

        List<Account> accounts = [
            SELECT Id, Description
            FROM Account
            WHERE Id IN :accountIds
        ];

        for (Account acc : accounts) {
```

```
        acc.Description = 'Processed';
    }

    update accounts;
}
}
```

24. Understand the code

Parameter

apex

```
List<Id> accountIds
```

means:

I'm receiving multiple Account Ids.

For example:

```
[001AAA, 001BBB, 001CCC]
```

Query

apex

```
WHERE Id IN :accountIds
```

means:

Give me all Accounts whose Id exists inside this list.

Loop

apex

```
for (Account acc : accounts) {
```

Go through each Account.

Then:

apex

```
acc.Description = 'Processed';
```

Change the Description.

Update

apex

```
update accounts;
```

Update all Accounts in one DML statement.

This is important Apex practice:

 Don't do:

apex

```
for (Account acc : accounts) {  
    update acc;  
}
```

Instead:

apex

```
for (Account acc : accounts) {  
    acc.Description = 'Processed';  
}  
  
update accounts;
```

That's bulkified.

25. Test Class for multiple Accounts

apex

```
@isTest  
public class AccountFutureProcessorTest {
```

```

@Test
static void testProcessAccounts() {

    List<Account> accounts = new List<Account>();

    for (Integer i = 0; i < 3; i++) {

        accounts.add(
            new Account(
                Name = 'Test Account ' + i
            )
        );
    }

    insert accounts;

    List<Id> accountIds = new List<Id>();

    for (Account acc : accounts) {
        accountIds.add(acc.Id);
    }

    Test.startTest();

    AccountFutureProcessor.processAccounts(accountIds);

    Test.stopTest();

    List<Account> updatedAccounts = [
        SELECT Id, Description
        FROM Account
        WHERE Id IN :accountIds
    ];

    for (Account acc : updatedAccounts) {

        System.assertEquals(
            'Processed',
            acc.Description
        );
    }
}

```

26. What's new in this test?

We created three Accounts:

apex

```
for (Integer i = 0; i < 3; i++)
```

Then:

apex

```
accounts.add(...)
```

adds them to our list.

Then:

apex

```
insert accounts;
```

inserts all three.

Then we create:

apex

```
List<Id> accountIds
```

and collect:

apex

```
acc.Id
```

for each Account.

Then:

apex

```
AccountFutureProcessor.processAccounts(accountIds);
```

passes all three IDs to the Future Method.

27. Important Future Method rules to memorize

For interview/coding questions, remember these.

Structure

apex

```
@future
public static void methodName(...) {

}
```

Rules

1. Must use `@future`

apex

```
@future
```

2. Must be `static`

apex

```
public static
```

3. Must return `void`

apex

```
void
```

4. Parameters should be primitive types or collections of primitive types

Examples:

apex

```
Id
String
Integer
Boolean
Decimal
List<Id>
Set<Id>
List<String>
```

5. Don't pass an sObject directly

Avoid:

apex

```
@future  
public static void process(Account acc)
```

Prefer:

apex

```
@future  
public static void process(Id accountId)
```

6. Use `Test.startTest()` and `Test.stopTest()` in tests

apex

```
Test.startTest();  
  
MyFutureClass.myMethod();  
  
Test.stopTest();
```

28. How you'll identify Future Method questions

If the question says something like:

"Create a method that should execute asynchronously..."

Think:

Future / Queueable

If it says:

"Make a callout asynchronously..."

Future can be used, provided you add:

apex

```
@future(callout=true)
```

For example:

apex

```
@future(callout=true)
public static void sendData(Id accountId) {

    // HTTP callout

}
```

The:

apex

```
callout=true
```

is specifically telling Salesforce:

"This Future Method is allowed to make an HTTP callout."

29. Very important: Future vs normal method

Normal

```
apex

public static void processAccount(Id accountId) {

    // executes synchronously

}
```

Future

```
apex

@future
public static void processAccount(Id accountId) {

    // executes asynchronously

}
```

The main difference is `@future`.

30. Your Future Method template

For your notes, memorize this:

apex

```
public class MyFutureClass {

    @future
    public static void processSomething(Id recordId) {

        // 1. Query record

        // 2. Perform processing

        // 3. Update record

    }

}
```

And test:

apex

```
@isTest
public class MyFutureClassTest {

    @isTest
    static void testProcessSomething() {

        // Create test data

        // Insert test data

        Test.startTest();

        // Call Future Method

        Test.stopTest();

        // Query result

        // Assert result

    }

}
```

★ The pattern you should remember

Future class:

```
@future
↓
static
↓
void
↓
parameters
↓
SOQL
↓
logic
↓
DML
```

Future test:

```
Create data
↓
Insert data
↓
Test.startTest()
↓
Call Future
↓
Test.stopTest()
↓
Query result
↓
System.assertEquals()
```

Next, **Queueable Apex** should be learned in exactly this same coding style, because you'll see a very important difference: instead of `@future`, you'll write `implements Queueable`, create an `execute()` method, use a constructor to pass data, and call it with `System.enqueueJob()`.